

Namespace ClippyCore.Agents

Classes

[Agent](#)

Base implementation of IAgent. Abstracts the use of AxControl from other clients. To create and dispose of Agents, use public AgentManager. [AgentManager](#)

[AgentManager](#)

Static class that properly Creates and Disposes of agents created at runtime. Abstracts other projects from creating Agent objects directly. Use this in projects to create Agents

[Animation](#)

Class with basic data about an Agent's available animations

[TtsAgent](#)

A type of IAgent that has TTS functionality. a TTSLd must be passed and is subsequently registered in AxControl. A list of common TTSLDs can be found in Global.TtsIDs.cs. NOTE: Not all .acs files have TTS capabilities. I will log some in this project but agents outside the scope of this will require trial and error. EX: Clippy does not have TTS, Bonzi Buddy does.

Enums

[AgentType](#)

Enum with values for each agent. When adding a new agent to make available, add an identifier here.

Class Agent

Namespace: [ClippyCore.Agents](#)

Assembly: ClippyCore.dll

Base implementation of IAgent. Abstracts the use of AxControl from other clients. To create and dispose of Agents, use public AgentManager. [AgentManager](#)

```
internal class Agent : IAgent
```

Inheritance

[object](#)  ← Agent

Implements

[IAgent](#)

Derived

[TtsAgent](#)

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

Agent(string, string, AgentType, AxControl)

```
public Agent(string acsPath, string agentName, AgentType agentType, AxControl axController)
```

Parameters

acsPath [string](#) 

agentName [string](#) 

agentType [AgentType](#)

axController AxControl

Properties

AcsPath

```
protected string AcsPath { get; private set; }
```

Property Value

[string](#)

AgentName

```
public string AgentName { get; private set; }
```

Property Value

[string](#)

AgentType

```
public AgentType AgentType { get; private set; }
```

Property Value

[AgentType](#)

Animations

```
public Dictionary<byte, Animation> Animations { get; private set; }
```

Property Value

[Dictionary](#) <[byte](#), [Animation](#)>

Controller

```
protected AxControl Controller { get; private set; }
```

Property Value

AxControl

Methods

GetAnimation(byte)

Returns Animation Obj based on its Animations key. Example usage:
GetAnimation(Animations.Bonzi.Aknowledge)

```
public Animation GetAnimation(byte key)
```

Parameters

key [byte](#) 

Key of animation in dictionary, as byte.

Returns

[Animation](#)

The Animation obj

GetAnimation(string)

Returns animation object based on its name.

```
public Animation GetAnimation(string animationName)
```

Parameters

animationName [string](#) 

Name of the animation obj to return/

Returns

[Animation](#)

Desired Animation obj

Exceptions

[ArgumentException](#)↗

[KeyNotFoundException](#)↗

GetAnimationNames()

Get a list of all available animations for the specific agent.

```
public string[] GetAnimationNames()
```

Returns

[string](#)↗[]

Animation names as strings in an array.

GetAnimations(bool)

Returns all available Animations for agent as dictionary.

```
public IDictionary GetAnimations(bool useStringKey = false)
```

Parameters

useStringKey [bool](#)↗

If true, returns the dictionary with string keys where the key is the animation name.

Returns

[Dictionary](#)

Dictionary where values are all Animation objects.

Hide()

Hides the agent from the screen, but is still running in background.

```
public void Hide()
```

MoveTo(short, short, int)

Moves agent to specified position. Use this version if knowingly passing 'shorts'. AxControl uses shorts as X,Y arguments natively.

```
public void MoveTo(short x, short y, int speed = 0)
```

Parameters

x [short](#)

X position relative to 0,0

y [short](#)

Y position relative to 0,0

speed [int](#)

Speed at which agent moves in milliseconds. 0 is instant.

MoveTo(int, int, int)

Moves agent to specified position. Use this version if using int values, like would be the case when using Screen.PrimaryScreen.Bounds or Windows.Forms Controls or Rectangles

```
public void MoveTo(int x, int y, int speed = 0)
```

Parameters

x [int](#)

X position relative to 0,0

y [int](#)

Y position relative to 0,0

speed [int](#)

Speed at which agent moves in miliseconds. 0 is instant.

Play(Animation)

Plays an animation by passing an Animation instance.

```
public void Play(Animation animation)
```

Parameters

animation [Animation](#)

The Animation

Play(byte)

Play an animation by passing a byte. Example usage: `_bonzi.Play(Animations.Bonzi.Aknowledge)`

```
public void Play(byte animationKey)
```

Parameters

animationKey [byte](#)

Play(string)

Plays an animation by passing a string (if you know it). You can call `GetAnimationNames()` to get a full list of available animations for each agent.

```
public void Play(string animation)
```

Parameters

animation [string](#)

Name of the animation.

Show()

Shows the agent on the screen.

```
public void Show()
```

Speak(Dictionary<string, string>, string)

Make agent speak specific phrase in a dictionary.

```
public void Speak(Dictionary<string, string> phrases, string key)
```

Parameters

phrases [Dictionary](#) <[string](#), [string](#)>

Phrase dictionary.

key [string](#)

Key of phrase value to speak

Speak(Dictionary<uint, string>, uint)

Make agent speak specific phrase in a dictionary.


```
public void Speak(Dictionary<uint, string> phrases, uint key)
```

Parameters

phrases [Dictionary](#) <[uint](#), [string](#)>

Phrase dictionary.

key [uint](#)

Key of phrase value to speak

Speak(List<string>)

Make the agent say multiple things in sequence. Can be used with Dictionaries of TKey,string by doing phrases.Values.ToList etc...

```
public void Speak(List<string> phrases)
```

Parameters

phrases [List](#) <[string](#)>

List of phrases for the agent to speak, one at a time.

Speak(string)

Makes the agent speak (speech bubble).

```
public void Speak(string phrase)
```

Parameters

phrase [string](#)

What the agent will say.

Speak(string[])

Make the agent say multiple things in sequence. Can be used with Dictionaries of TKey,string using phrases.Values.ToArray etc...

```
public void Speak(string[] phrases)
```

Parameters

phrases [string](#)[]

Array of string for the agent to speak.

Stop()

Force stop an Agent animation.

```
public void Stop()
```

Class AgentManager

Namespace: [ClippyCore.Agents](#)

Assembly: ClippyCore.dll

Static class that properly Creates and Disposes of agents created at runtime. Abstracts other projects from creating Agent objects directly. Use this in projects to create Agents

```
public static class AgentManager
```

Inheritance

[object](#)  ← AgentManager

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

CreateAgent(AxControl, AgentType)

Creates and agent. Multiple agents can be created and used at any given time using one axControl.

```
public static IAgent CreateAgent(AxControl axControl, AgentType agentType)
```

Parameters

axControl AxControl

AxControl object the agent will use (should only ever be one at any given time)

agentType [AgentType](#)

Enum used to determine what values to set when instantiating a new Agent.

Returns

[IAgent](#)

A class that implements IAgent, to be used during program execution.

Enum AgentType

Namespace: [ClippyCore.Agents](#)

Assembly: ClippyCore.dll

Enum with values for each agent. When adding a new agent to make available, add an identifier here.

```
public enum AgentType
```

Fields

```
Bonzi = 1
```

```
Clippy = 0
```

```
Shrek = 2
```

Class Animation

Namespace: [ClippyCore.Agents](#)

Assembly: ClippyCore.dll

Class with basic data about an Agent's available animations

```
public class Animation
```

Inheritance

[object](#)  ← Animation

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

Animation(string, string)

```
private Animation(string name, string displayName = null)
```

Parameters

name [string](#) 

displayName [string](#) 

Properties

DisplayName

```
public string DisplayName { get; }
```

Property Value

[string](#)

Name

```
public string Name { get; }
```

Property Value

[string](#)

Methods

Create(string, IAgent, string)

Factory to create new Animations.

```
public static Animation Create(string name, IAgent agent, string displayName = null)
```

Parameters

name [string](#)

Name of the animation

agent [IAgent](#)

Instance of Agent to attach animations to.

displayName [string](#)

Returns

[Animation](#)

The newly instantiated Animation

Play(IAgent)

Extra way to play an animation. Not super useful but there anyways if needed. Could be used if you're crafty and have multiple Agents interacting with each other or if some Agents share common animations... Ex: var myAnim = Animation.Create("Aknowledge", _bonzi); myAnim.Play(_bonzi);

```
public void Play(IAgent agent)
```

Parameters

agent [IAgent](#)

Agent to perform the animation.

Class TtsAgent

Namespace: [ClippyCore.Agents](#)

Assembly: ClippyCore.dll

A type of IAgent that has TTS functionality. a TTSId must be passed and is subsequently registered in AxControl. A list of common TTSIds can be found in Global.TtsIDs.cs. NOTE: Not all .acs files have TTS capabilities. I will log some in this project but agents outside the scope of this will require trial and error. EX: Clippy does not have TTS, Bonzi Buddy does.

```
internal class TtsAgent : Agent, IAgent
```

Inheritance

[object](#)  ← [Agent](#) ← TtsAgent

Implements

[IAgent](#)

Inherited Members

[Agent.AgentName](#) , [Agent.AgentType](#) , [Agent.Animations](#) , [Agent.AcsPath](#) , [Agent.Controller](#) , [Agent.Hide\(\)](#) , [Agent.MoveTo\(int, int, int\)](#) , [Agent.MoveTo\(short, short, int\)](#) , [Agent.Play\(string\)](#) , [Agent.Play\(Animation\)](#) , [Agent.Play\(byte\)](#) , [Agent.Show\(\)](#) , [Agent.Speak\(string\)](#) , [Agent.Speak\(List<string>\)](#) , [Agent.Speak\(string\[\]\)](#) , [Agent.Speak\(Dictionary<string, string>, string\)](#) , [Agent.Speak\(Dictionary<uint, string>, uint\)](#) , [Agent.Stop\(\)](#) , [Agent.GetAnimationNames\(\)](#) , [Agent.GetAnimations\(bool\)](#) , [Agent.GetAnimation\(string\)](#) , [Agent.GetAnimation\(byte\)](#) , [object.Equals\(object\) !\[\]\(cf531ed27e91483460120fcc057b3901_img.jpg\)](#) , [object.Equals\(object, object\) !\[\]\(34fde9b7c74442c0438f550a41236260_img.jpg\)](#) , [object.GetHashCode\(\) !\[\]\(f3ffd03e145adb5d0f6f54d9f4fb82fd_img.jpg\)](#) , [object.GetType\(\) !\[\]\(1512695720264d2aab11e6ec2cb67c0e_img.jpg\)](#) , [object.MemberwiseClone\(\) !\[\]\(d06e0e0a5ce4085deb80bf730d27022b_img.jpg\)](#) , [object.ReferenceEquals\(object, object\) !\[\]\(6c9aeb30cbfe3eb949d668eaafd2ff33_img.jpg\)](#) , [object.ToString\(\) !\[\]\(f74c242d3b0eaa1097039d595582d93a_img.jpg\)](#)

Constructors

TtsAgent(string, string, AgentType, AxControl, string)

```
public TtsAgent(string acsPath, string agentName, AgentType agentType, AxControl controller, string ttsID)
```

Parameters

acsPath [string](#) 

agentName [string](#)

agentType [AgentType](#)

controller AxControl

ttsID [string](#)

Properties

TtsID

```
public string TtsID { get; private set; }
```

Property Value

[string](#)

Namespace ClippyCore.Data

Classes

[Keys](#)

Class Keys

Namespace: [ClippyCore.Data](#)

Assembly: ClippyCore.dll

```
internal static class Keys
```

Inheritance

[object](#)  ← Keys

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Fields

GoogleAi

```
internal const string GoogleAi = "AIzaSyCfTcY5wlgRHUgvBPiCi5PCVrmraKf1h1E"
```

Field Value

[string](#) 

Namespace ClippyCore.Global

Classes

[AgentNames](#)

Static string with available agents' names.

[Animations](#)

Keys for the IAgent animation dictionary mapping. Keys are always bytes.

[Animations.Bonzi](#)

All animation keys for the Bonzi agent.

[Animations.Clippy](#)

All animation keys for Clippy agent.

[Animations.Shrek](#)

All animation keys for Shrek agent.

[Paths](#)

Paths to ACS Files and data files.

[TtsIDs](#)

TTSModelIDs for agents with TTS capabilities.

Class AgentNames

Namespace: [ClippyCore.Global](#)

Assembly: ClippyCore.dll

Static string with available agents' names.

```
internal static class AgentNames
```

Inheritance

[object](#)  ← AgentNames

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Fields

BONZI

```
public const string BONZI = "Bonzi Vine"
```

Field Value

[string](#) 

CLIPPY

```
public const string CLIPPY = "Clippit"
```

Field Value

[string](#) 

SHREK

```
public const string SHREK = "Shrek"
```

Field Value

[string](#) 

Class Animations

Namespace: [ClippyCore.Global](#)

Assembly: ClippyCore.dll

Keys for the IAgent animation dictionary mapping. Keys are always bytes.

```
public static class Animations
```

Inheritance

[object](#)  ← Animations

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Class Animations.Bonzi

Namespace: [ClippyCore.Global](#)

Assembly: ClippyCore.dll

All animation keys for the Bonzi agent.

```
public static class Animations.Bonzi
```

Inheritance

[object](#)  ← Animations.Bonzi

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Fields

Acknowledge

```
public const byte Acknowledge = 0
```

Field Value

[byte](#) 

Alert

```
public const byte Alert = 1
```

Field Value

[byte](#) 

Banana

```
public const byte Banana = 2
```

Field Value

[byte](#)

BananaMiss

```
public const byte BananaMiss = 3
```

Field Value

[byte](#)

Blink

```
public const byte Blink = 4
```

Field Value

[byte](#)

BlowKiss

```
public const byte BlowKiss = 5
```

Field Value

[byte](#)

Butternut

```
public const byte Butternut = 6
```

Field Value

[byte](#)

Confused

```
public const byte Confused = 7
```

Field Value

[byte](#)

Congratulate

```
public const byte Congratulate = 8
```

Field Value

[byte](#)

Decline

```
public const byte Decline = 9
```

Field Value

[byte](#)

DoMagic1

```
public const byte DoMagic1 = 10
```

Field Value

[byte](#)

DoMagic2

```
public const byte DoMagic2 = 11
```

Field Value

[byte](#)

DontRecognize

```
public const byte DontRecognize = 12
```

Field Value

[byte](#)

Explain

```
public const byte Explain = 13
```

Field Value

[byte](#)

Explain2

```
public const byte Explain2 = 14
```

Field Value

[byte](#)

Explain3

```
public const byte Explain3 = 15
```

Field Value

[byte](#)

Explain4

```
public const byte Explain4 = 16
```

Field Value

[byte](#)

GestureDown

```
public const byte GestureDown = 17
```

Field Value

[byte](#)

GestureLeft

```
public const byte GestureLeft = 18
```

Field Value

[byte](#)

GestureRight

```
public const byte GestureRight = 19
```

Field Value

[byte](#)

GestureUp

```
public const byte GestureUp = 20
```

Field Value

[byte](#)

GetAttention

```
public const byte GetAttention = 21
```

Field Value

[byte](#)

GetAttention2

```
public const byte GetAttention2 = 22
```

Field Value

[byte](#)

GetAttentionContinued

```
public const byte GetAttentionContinued = 23
```

Field Value

[byte](#)

GetAttentionReturn

```
public const byte GetAttentionReturn = 24
```

Field Value

[byte](#)

Giggle

```
public const byte Giggle = 25
```

Field Value

[byte](#)

Greet

```
public const byte Greet = 26
```

Field Value

[byte](#)

HandsBehind

```
public const byte HandsBehind = 27
```

Field Value

[byte](#)

HeadphonesContinued

```
public const byte HeadphonesContinued = 28
```

Field Value

[byte](#)

HeadphonesReturn

```
public const byte HeadphonesReturn = 29
```

Field Value

[byte](#)

Hearing1


```
public const byte Hearing1 = 30
```

Field Value

[byte](#)

Hearing2

```
public const byte Hearing2 = 31
```

Field Value

[byte](#)

Hearing3

```
public const byte Hearing3 = 32
```

Field Value

[byte](#)

Hide

```
public const byte Hide = 33
```

Field Value

[byte](#)

HideShow

```
public const byte HideShow = 34
```

Field Value

[byte](#)

Hug

```
public const byte Hug = 35
```

Field Value

[byte](#)

Idle1_1

```
public const byte Idle1_1 = 36
```

Field Value

[byte](#)

Idle1_11

```
public const byte Idle1_11 = 39
```

Field Value

[byte](#)

Idle1_12

```
public const byte Idle1_12 = 40
```

Field Value

[byte](#)

Idle1_13

```
public const byte Idle1_13 = 41
```

Field Value

[byte](#)

Idle1_14

```
public const byte Idle1_14 = 42
```

Field Value

[byte](#)

Idle1_15

```
public const byte Idle1_15 = 43
```

Field Value

[byte](#)

Idle1_1_2

```
public const byte Idle1_1_2 = 37
```

Field Value

[byte](#)

Idle1_1_3

```
public const byte Idle1_1_3 = 38
```

Field Value

[byte](#)

Idle1_20

```
public const byte Idle1_20 = 44
```

Field Value

[byte](#)

Idle1_21

```
public const byte Idle1_21 = 45
```

Field Value

[byte](#)

Idle1_22

```
public const byte Idle1_22 = 46
```

Field Value

[byte](#)

Idle1_24

```
public const byte Idle1_24 = 47
```

Field Value

[byte](#)

Idle1_25

```
public const byte Idle1_25 = 48
```

Field Value

[byte](#)

Idle1_26

```
public const byte Idle1_26 = 49
```

Field Value

[byte](#)

Idle1_3

```
public const byte Idle1_3 = 50
```

Field Value

[byte](#)

Idle1_4

```
public const byte Idle1_4 = 51
```

Field Value

[byte](#)

Idle1_4_2

```
public const byte Idle1_4_2 = 52
```

Field Value

[byte](#)

Idle1_5

```
public const byte Idle1_5 = 53
```

Field Value

[byte](#)

Idle1_5_2

```
public const byte Idle1_5_2 = 54
```

Field Value

[byte](#)

Idle1_6

```
public const byte Idle1_6 = 55
```

Field Value

[byte](#)

Idle1_7

```
public const byte Idle1_7 = 56
```

Field Value

[byte](#)

Idle1_8

```
public const byte Idle1_8 = 57
```

Field Value

[byte](#)

Idle1_9

```
public const byte Idle1_9 = 58
```

Field Value

[byte](#)

Idle1_9_2

```
public const byte Idle1_9_2 = 59
```

Field Value

[byte](#)

Idle1_9_3

```
public const byte Idle1_9_3 = 60
```

Field Value

[byte](#)

Idle3_1

```
public const byte Idle3_1 = 61
```

Field Value

[byte](#)

Idle3_2


```
public const byte Idle3_2 = 62
```

Field Value

[byte](#)

Idle3_3

```
public const byte Idle3_3 = 63
```

Field Value

[byte](#)

Juggle

```
public const byte Juggle = 64
```

Field Value

[byte](#)

LookDown

```
public const byte LookDown = 65
```

Field Value

[byte](#)

LookDownBlink

```
public const byte LookDownBlink = 66
```

Field Value

[byte](#)

LookDownLeft

```
public const byte LookDownLeft = 67
```

Field Value

[byte](#)

LookDownLeftBlink

```
public const byte LookDownLeftBlink = 68
```

Field Value

[byte](#)

LookDownLeftReturn

```
public const byte LookDownLeftReturn = 69
```

Field Value

[byte](#)

LookDownReturn

```
public const byte LookDownReturn = 70
```

Field Value

[byte](#)

LookDownRight

```
public const byte LookDownRight = 71
```

Field Value

[byte](#)

LookDownRightBlink

```
public const byte LookDownRightBlink = 72
```

Field Value

[byte](#)

LookDownRightReturn

```
public const byte LookDownRightReturn = 73
```

Field Value

[byte](#)

LookLeft

```
public const byte LookLeft = 74
```

Field Value

[byte](#)

LookLeftBlink

```
public const byte LookLeftBlink = 75
```

Field Value

[byte](#)

LookLeftReturn

```
public const byte LookLeftReturn = 76
```

Field Value

[byte](#)

LookRight

```
public const byte LookRight = 77
```

Field Value

[byte](#)

LookRightBlink

```
public const byte LookRightBlink = 78
```

Field Value

[byte](#)

LookRightReturn

```
public const byte LookRightReturn = 79
```

Field Value

[byte](#)

LookUp

```
public const byte LookUp = 80
```

Field Value

[byte](#)

LookUpBlink

```
public const byte LookUpBlink = 81
```

Field Value

[byte](#)

LookUpLeft

```
public const byte LookUpLeft = 82
```

Field Value

[byte](#)

LookUpLeftBlink

```
public const byte LookUpLeftBlink = 83
```

Field Value

[byte](#)

LookUpLeftReturn

```
public const byte LookUpLeftReturn = 84
```

Field Value

[byte](#)

LookUpReturn

```
public const byte LookUpReturn = 85
```

Field Value

[byte](#)

LookUpRight

```
public const byte LookUpRight = 86
```

Field Value

[byte](#)

LookUpRightBlink

```
public const byte LookUpRightBlink = 87
```

Field Value

[byte](#)

LookUpRightReturn

```
public const byte LookUpRightReturn = 88
```

Field Value

[byte](#)

MailCheck

```
public const byte MailCheck = 89
```

Field Value

[byte](#)

MailCheckEmpty

```
public const byte MailCheckEmpty = 90
```

Field Value

[byte](#)

MailCheckFull

```
public const byte MailCheckFull = 91
```

Field Value

[byte](#)

MailNext

```
public const byte MailNext = 92
```

Field Value

[byte](#)

MailRead

```
public const byte MailRead = 93
```

Field Value

[byte](#)

MailReturn


```
public const byte MailReturn = 94
```

Field Value

[byte](#)

MoveDown

```
public const byte MoveDown = 95
```

Field Value

[byte](#)

MoveDownReturn

```
public const byte MoveDownReturn = 96
```

Field Value

[byte](#)

MoveLeft

```
public const byte MoveLeft = 97
```

Field Value

[byte](#)

MoveLeftReturn

```
public const byte MoveLeftReturn = 98
```

Field Value

[byte](#)

MoveRight

```
public const byte MoveRight = 99
```

Field Value

[byte](#)

MoveRightReturn

```
public const byte MoveRightReturn = 100
```

Field Value

[byte](#)

MoveUp

```
public const byte MoveUp = 101
```

Field Value

[byte](#)

MoveUpReturn

```
public const byte MoveUpReturn = 102
```

Field Value

[byte](#)

Pleased

```
public const byte Pleased = 103
```

Field Value

[byte](#)

PleasedHard

```
public const byte PleasedHard = 104
```

Field Value

[byte](#)

PleasedSoft

```
public const byte PleasedSoft = 105
```

Field Value

[byte](#)

Process

```
public const byte Process = 106
```

Field Value

[byte](#)

Processing

```
public const byte Processing = 107
```

Field Value

[byte](#)

Read

```
public const byte Read = 108
```

Field Value

[byte](#)

ReadContinued

```
public const byte ReadContinued = 109
```

Field Value

[byte](#)

ReadLookUp

```
public const byte ReadLookUp = 111
```

Field Value

[byte](#)

ReadLookUpContinued

```
public const byte ReadLookUpContinued = 112
```

Field Value

[byte](#)

ReadLookUpReturn

```
public const byte ReadLookUpReturn = 113
```

Field Value

[byte](#)

ReadReturn

```
public const byte ReadReturn = 114
```

Field Value

[byte](#)

Reading

```
public const byte Reading = 110
```

Field Value

[byte](#)

RestPose

```
public const byte RestPose = 115
```

Field Value

[byte](#)

Sad

```
public const byte Sad = 116
```

Field Value

[byte](#)

Scout

```
public const byte Scout = 117
```

Field Value

[byte](#)

ScoutAlert

```
public const byte ScoutAlert = 118
```

Field Value

[byte](#)

ScoutLeft

```
public const byte ScoutLeft = 119
```

Field Value

[byte](#)

ScoutReturn

```
public const byte ScoutReturn = 120
```

Field Value

[byte](#)

ScoutRight

```
public const byte ScoutRight = 121
```

Field Value

[byte](#)

Search

```
public const byte Search = 122
```

Field Value

[byte](#)

Searching

```
public const byte Searching = 123
```

Field Value

[byte](#)

SearchingReturn

```
public const byte SearchingReturn = 124
```

Field Value

[byte](#)

Shoosh

```
public const byte Shoosh = 125
```

Field Value

[byte](#)

Show


```
public const byte Show = 126
```

Field Value

[byte](#)

StartListening

```
public const byte StartListening = 127
```

Field Value

[byte](#)

StopListening

```
public const byte StopListening = 128
```

Field Value

[byte](#)

Suggest

```
public const byte Suggest = 129
```

Field Value

[byte](#)

SunglassesContinued

```
public const byte SunglassesContinued = 130
```

Field Value

[byte](#)

SunglassesReturn

```
public const byte SunglassesReturn = 131
```

Field Value

[byte](#)

Surprised

```
public const byte Surprised = 132
```

Field Value

[byte](#)

Think

```
public const byte Think = 133
```

Field Value

[byte](#)

Thinking

```
public const byte Thinking = 134
```

Field Value

[byte](#)

Unbelievable

```
public const byte Unbelievable = 135
```

Field Value

[byte](#)

Uncertain

```
public const byte Uncertain = 136
```

Field Value

[byte](#)

Wave

```
public const byte Wave = 137
```

Field Value

[byte](#)

Wink

```
public const byte Wink = 138
```

Field Value

[byte](#)

Write

```
public const byte Write = 139
```

Field Value

[byte](#)

WriteContinued

```
public const byte WriteContinued = 140
```

Field Value

[byte](#)

WriteOnce

```
public const byte WriteOnce = 141
```

Field Value

[byte](#)

WriteOnceAgain

```
public const byte WriteOnceAgain = 142
```

Field Value

[byte](#)

WritePause

```
public const byte WritePause = 143
```

Field Value

[byte](#)

WritePre

```
public const byte WritePre = 144
```

Field Value

[byte](#)

WriteReturn

```
public const byte WriteReturn = 145
```

Field Value

[byte](#)

Writing

```
public const byte Writing = 146
```

Field Value

[byte](#)

WritingReturn

```
public const byte WritingReturn = 147
```

Field Value

[byte](#)

Class Animations.Clippy

Namespace: [ClippyCore.Global](#)

Assembly: ClippyCore.dll

All animation keys for Clippy agent.

```
public static class Animations.Clippy
```

Inheritance

[object](#)  ← Animations.Clippy

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Fields

Alert

```
public const byte Alert = 0
```

Field Value

[byte](#) 

CheckingSomething

```
public const byte CheckingSomething = 1
```

Field Value

[byte](#) 

Congratulate

```
public const byte Congratulate = 2
```

Field Value

[byte](#)

EmptyTrash

```
public const byte EmptyTrash = 3
```

Field Value

[byte](#)

Explain

```
public const byte Explain = 4
```

Field Value

[byte](#)

GestureDown

```
public const byte GestureDown = 5
```

Field Value

[byte](#)

GestureLeft


```
public const byte GestureLeft = 6
```

Field Value

[byte](#)

GestureRight

```
public const byte GestureRight = 7
```

Field Value

[byte](#)

GestureUp

```
public const byte GestureUp = 8
```

Field Value

[byte](#)

GetArtsy

```
public const byte GetArtsy = 9
```

Field Value

[byte](#)

GetAttention

```
public const byte GetAttention = 10
```

Field Value

[byte](#)

GetTechy

```
public const byte GetTechy = 11
```

Field Value

[byte](#)

GetWizardy

```
public const byte GetWizardy = 12
```

Field Value

[byte](#)

GoodBye

```
public const byte GoodBye = 13
```

Field Value

[byte](#)

Greeting

```
public const byte Greeting = 14
```

Field Value

[byte](#)

Hearing_1

```
public const byte Hearing_1 = 15
```

Field Value

[byte](#)

Hide

```
public const byte Hide = 16
```

Field Value

[byte](#)

Idle1_1

```
public const byte Idle1_1 = 17
```

Field Value

[byte](#)

IdleAtom

```
public const byte IdleAtom = 18
```

Field Value

[byte](#)

IdleEyeBrowRaise

```
public const byte IdleEyeBrowRaise = 19
```

Field Value

[byte](#)

IdleFingerTap

```
public const byte IdleFingerTap = 20
```

Field Value

[byte](#)

IdleHeadScratch

```
public const byte IdleHeadScratch = 21
```

Field Value

[byte](#)

IdleRopePile

```
public const byte IdleRopePile = 22
```

Field Value

[byte](#)

IdleSideToSide

```
public const byte IdleSideToSide = 23
```

Field Value

[byte](#)

IdleSnooze

```
public const byte IdleSnooze = 24
```

Field Value

[byte](#)

LookDown

```
public const byte LookDown = 25
```

Field Value

[byte](#)

LookDownLeft

```
public const byte LookDownLeft = 26
```

Field Value

[byte](#)

LookDownRight

```
public const byte LookDownRight = 27
```

Field Value

[byte](#)

LookLeft

```
public const byte LookLeft = 28
```

Field Value

[byte](#)

LookRight

```
public const byte LookRight = 29
```

Field Value

[byte](#)

LookUp

```
public const byte LookUp = 30
```

Field Value

[byte](#)

LookUpLeft

```
public const byte LookUpLeft = 31
```

Field Value

[byte](#)

LookUpRight

```
public const byte LookUpRight = 32
```

Field Value

[byte](#)

Print

```
public const byte Print = 33
```

Field Value

[byte](#)

Processing

```
public const byte Processing = 34
```

Field Value

[byte](#)

RestPose

```
public const byte RestPose = 35
```

Field Value

[byte](#)

Save

```
public const byte Save = 36
```

Field Value

[byte](#)

Searching

```
public const byte Searching = 37
```

Field Value

[byte](#)

SendMail


```
public const byte SendMail = 38
```

Field Value

[byte](#)

Show

```
public const byte Show = 39
```

Field Value

[byte](#)

Thinking

```
public const byte Thinking = 40
```

Field Value

[byte](#)

Wave

```
public const byte Wave = 41
```

Field Value

[byte](#)

Writing

```
public const byte Writing = 42
```

Field Value

[byte](#)

Class Animations.Shrek

Namespace: [ClippyCore.Global](#)

Assembly: ClippyCore.dll

All animation keys for Shrek agent.

```
public static class Animations.Shrek
```

Inheritance

[object](#)  ← Animations.Shrek

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Fields

Back

```
public const byte Back = 0
```

Field Value

[byte](#) 

BeAGoodFriend

```
public const byte BeAGoodFriend = 1
```

Field Value

[byte](#) 

BookRead

```
public const byte BookRead = 2
```

Field Value

[byte](#)

Burp

```
public const byte Burp = 3
```

Field Value

[byte](#)

BuyTwoAndThird

```
public const byte BuyTwoAndThird = 4
```

Field Value

[byte](#)

ByeByeSeeYouNextTime

```
public const byte ByeByeSeeYouNextTime = 5
```

Field Value

[byte](#)

ChangeYourSettings

```
public const byte ChangeYourSettings = 6
```

Field Value

[byte](#)

CheckYourTukiMail

```
public const byte CheckYourTukiMail = 7
```

Field Value

[byte](#)

CleanYourRoom

```
public const byte CleanYourRoom = 8
```

Field Value

[byte](#)

ClubTuki

```
public const byte ClubTuki = 9
```

Field Value

[byte](#)

DontForgetToEmailYourParents

```
public const byte DontForgetToEmailYourParents = 10
```

Field Value

[byte](#)

DownloadMoreThemes

```
public const byte DownloadMoreThemes = 11
```

Field Value

[byte](#)

EnterWebAddress

```
public const byte EnterWebAddress = 12
```

Field Value

[byte](#)

Exit

```
public const byte Exit = 13
```

Field Value

[byte](#)

Farting

```
public const byte Farting = 14
```

Field Value

[byte](#)

FeedYourPet

```
public const byte FeedYourPet = 15
```

Field Value

[byte](#)

Forward

```
public const byte Forward = 16
```

Field Value

[byte](#)

GesturingDown

```
public const byte GesturingDown = 17
```

Field Value

[byte](#)

GesturingDownReturn

```
public const byte GesturingDownReturn = 18
```

Field Value

[byte](#)

GesturingLeft

```
public const byte GesturingLeft = 19
```

Field Value

[byte](#)

GesturingLeftReturn

```
public const byte GesturingLeftReturn = 20
```

Field Value

[byte](#)

GesturingRight

```
public const byte GesturingRight = 21
```

Field Value

[byte](#)

GesturingRightReturn


```
public const byte GesturingRightReturn = 22
```

Field Value

[byte](#)

GesturingUp

```
public const byte GesturingUp = 23
```

Field Value

[byte](#)

GesturingUpReturn

```
public const byte GesturingUpReturn = 24
```

Field Value

[byte](#)

GetSomeHelp

```
public const byte GetSomeHelp = 25
```

Field Value

[byte](#)

GoodAfternoon

```
public const byte GoodAfternoon = 26
```

Field Value

[byte](#)

GreatToSeeYouMorning

```
public const byte GreatToSeeYouMorning = 27
```

Field Value

[byte](#)

Hiding

```
public const byte Hiding = 28
```

Field Value

[byte](#)

Home

```
public const byte Home = 29
```

Field Value

[byte](#)

IHopeYouAteBreakfast

```
public const byte IHopeYouAteBreakfast = 32
```

Field Value

[byte](#)

ILoveGettingEmailRats

```
public const byte ILoveGettingEmailRats = 34
```

Field Value

[byte](#)

ILovegettingEmailDonkey

```
public const byte ILovegettingEmailDonkey = 33
```

Field Value

[byte](#)

IfYouReallyWantToVisit

```
public const byte IfYouReallyWantToVisit = 30
```

Field Value

[byte](#)

IfYouWantToBuyCD

```
public const byte IfYouWantToBuyCD = 31
```

Field Value

[byte](#)

LaunchNewWindow

```
public const byte LaunchNewWindow = 35
```

Field Value

[byte](#)

LearnEarnWin

```
public const byte LearnEarnWin = 36
```

Field Value

[byte](#)

MovingDown

```
public const byte MovingDown = 37
```

Field Value

[byte](#)

MovingLeft

```
public const byte MovingLeft = 38
```

Field Value

[byte](#)

MovingReturn

```
public const byte MovingReturn = 39
```

Field Value

[byte](#)

MovingRight

```
public const byte MovingRight = 40
```

Field Value

[byte](#)

MovingUp

```
public const byte MovingUp = 41
```

Field Value

[byte](#)

OhYourAWinner

```
public const byte OhYourAWinner = 42
```

Field Value

[byte](#)

PrintThisPage

```
public const byte PrintThisPage = 43
```

Field Value

[byte](#)

ReferAFriend

```
public const byte ReferAFriend = 44
```

Field Value

[byte](#)

Refresh

```
public const byte Refresh = 45
```

Field Value

[byte](#)

RememberIAmHereToHelp

```
public const byte RememberIAmHereToHelp = 46
```

Field Value

[byte](#)

RememberToBrushYourTeeth

```
public const byte RememberToBrushYourTeeth = 47
```

Field Value

[byte](#)

RememberToGiveDadHug

```
public const byte RememberToGiveDadHug = 48
```

Field Value

[byte](#)

RememberToGiveMomHug

```
public const byte RememberToGiveMomHug = 49
```

Field Value

[byte](#)

RememberToPickUpToys

```
public const byte RememberToPickUpToys = 50
```

Field Value

[byte](#)

RememberToShare

```
public const byte RememberToShare = 51
```

Field Value

[byte](#)

RememberYourParents

```
public const byte RememberYourParents = 52
```

Field Value

[byte](#)

Showing

```
public const byte Showing = 53
```

Field Value

[byte](#)

Stop


```
public const byte Stop = 54
```

Field Value

[byte](#)

ToiletFlush

```
public const byte ToiletFlush = 55
```

Field Value

[byte](#)

TriviaBathTub

```
public const byte TriviaBathTub = 56
```

Field Value

[byte](#)

TriviaFairyTale

```
public const byte TriviaFairyTale = 57
```

Field Value

[byte](#)

TriviaMikeMyers

```
public const byte TriviaMikeMyers = 58
```

Field Value

[byte](#)

TriviaPeanutButter

```
public const byte TriviaPeanutButter = 59
```

Field Value

[byte](#)

TriviaShrekManHours

```
public const byte TriviaShrekManHours = 60
```

Field Value

[byte](#)

TriviaWashHair

```
public const byte TriviaWashHair = 61
```

Field Value

[byte](#)

Tuki

```
public const byte Tuki = 62
```

Field Value

[byte](#)

TukiMoola

```
public const byte TukiMoola = 63
```

Field Value

[byte](#)

TukiTukiTuki

```
public const byte TukiTukiTuki = 64
```

Field Value

[byte](#)

TuuuuKiiii

```
public const byte TuuuuKiiii = 65
```

Field Value

[byte](#)

WhatAWonderfulAfternoon

```
public const byte WhatAWonderfulAfternoon = 66
```

Field Value

[byte](#)

WhatAWonderfulEvening

```
public const byte WhatAWonderfulEvening = 67
```

Field Value

[byte](#)

YourFamilyLovesYou

```
public const byte YourFamilyLovesYou = 68
```

Field Value

[byte](#)

Class Paths

Namespace: [ClippyCore.Global](#)

Assembly: ClippyCore.dll

Paths to ACS Files and data files.

```
internal static class Paths
```

Inheritance

[object](#)  ← Paths

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Properties

BONZI

```
public static string BONZI { get; }
```

Property Value

[string](#) 

CLIPPY

```
public static string CLIPPY { get; }
```

Property Value

[string](#) 

LOGGER

```
public static string LOGGER { get; }
```

Property Value

[string](#)

SHREK

```
public static string SHREK { get; }
```

Property Value

[string](#)

Class TtsIDs

Namespace: [ClippyCore.Global](#)

Assembly: ClippyCore.dll

TTSModelIDs for agents with TTS capabilities.

```
internal static class TtsIDs
```

Inheritance

[object](#)  ← TtsIDs

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Fields

BONZI

```
public const string BONZI = "{CA141FD0-AC7F-11D1-97A3-006008273001}"
```

Field Value

[string](#) 

Namespace ClippyCore.Helpers

Classes

[Logger](#)

This static class is responsible for logging warning and errors to logs.txt

Class Logger

Namespace: [ClippyCore.Helpers](#)

Assembly: ClippyCore.dll

This static class is responsible for logging warning and errors to logs.txt

```
public static class Logger
```

Inheritance

[object](#)  ← Logger

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

Log(string)

Log a message to logs.txt

```
public static void Log(string message)
```

Parameters

message [string](#) 

Message to log.

LogAndThrow(string, Exception)

Log a message to logs.txt AND throw the error.

```
public static void LogAndThrow(string message, Exception e)
```

Parameters

`message` [string](#) 

Message to log.

`e` [Exception](#) 

Error/Exception to throw.

Namespace ClippyCore.Interfaces

Interfaces

[IAgent](#)

Interface contract for all Agents and derived classes. Agent implements IAgent, and other derivatives inherit from Agent Base implementation is Agent.cs

Interface IAgent

Namespace: [ClippyCore.Interfaces](#)

Assembly: ClippyCore.dll

Interface contract for all Agents and derived classes. Agent implements IAgent, and other derivatives inherit from Agent Base implementation is Agent.cs

```
public interface IAgent
```

Methods

GetAnimation(byte)

```
Animation GetAnimation(byte key)
```

Parameters

key [byte](#) 

Returns

[Animation](#)

GetAnimation(string)

```
Animation GetAnimation(string animationName)
```

Parameters

animationName [string](#) 

Returns

[Animation](#)

GetAnimationNames()

```
string[] GetAnimationNames()
```

Returns

[string](#)[]

GetAnimations(bool)

```
IDictionary GetAnimations(bool useStringKey = false)
```

Parameters

useStringKey [bool](#)

Returns

[IDictionary](#)

Hide()

```
void Hide()
```

MoveTo(short, short, int)

```
void MoveTo(short x, short y, int speed = 0)
```

Parameters

x [short](#)

y [short](#)

speed [int](#)

MoveTo(int, int, int)

```
void MoveTo(int x, int y, int speed = 0)
```

Parameters

x [int](#)

y [int](#)

speed [int](#)

Play(Animation)

```
void Play(Animation animation)
```

Parameters

animation [Animation](#)

Play(byte)

```
void Play(byte animationKey)
```

Parameters

animationKey [byte](#)

Play(string)

```
void Play(string animation)
```

Parameters

animation [string](#)

Show()

```
void Show()
```

Speak(Dictionary<string, string>, string)

```
void Speak(Dictionary<string, string> phrases, string key)
```

Parameters

phrases [Dictionary](#) <[string](#), [string](#)>

key [string](#)

Speak(Dictionary<uint, string>, uint)

```
void Speak(Dictionary<uint, string> phrases, uint key)
```

Parameters

phrases [Dictionary](#) <[uint](#), [string](#)>

key [uint](#)

Speak(List<string>)

```
void Speak(List<string> phrases)
```

Parameters

phrases [List](#) <[string](#)>

Speak(string)

```
void Speak(string phrase)
```

Parameters

phrase [string](#) 

Speak(string[])

```
void Speak(string[] phrases)
```

Parameters

phrases [string](#)  []

Stop()

```
void Stop()
```